

Comprehensive Build Size Optimization Guide (Sub-10MB Targets)

For React Native & ViroReact (Android Instant Apps & iOS App Clips)

This guide is designed for developers of all experience levels (including junior developers) to help you understand, configure, and compile highly optimized, lightweight versions of your Augmented Reality (AR) application.

By default, standard React Native builds package everything into a single "universal" file that is very heavy (~150MB). To run as an **Android Instant App** or **iOS App Clip**, your application must be optimized to download in seconds over cellular networks. This guide will show you how to shrink your production download footprint to **under 10MB - 15MB**.

Table of Contents

1. [Core Concepts: What makes an app small?](#)
 2. [Android \(Instant Apps\) - Step-by-Step Optimization](#)
 3. [iOS \(App Clips\) - Step-by-Step Optimization](#)
 4. [JavaScript & Asset Optimizations \(Shared Code\)](#)
 5. [Summary Checklist for Release Day](#)
-

1. Core Concepts: What makes an app small?

Before changing any code, let's understand the three main concepts we are using to reduce app size:

1. **Architecture Splitting (ABI Splits):** Phones have different types of processors (CPUs). A "universal" build contains code for all CPU types in one file. "Splitting" means we compile separate apps for each CPU type. A user with an ARM64 processor only downloads the ARM64 code, saving up to 70% of the download size.
 2. **Code Minification & Obfuscation (Proguard / R8):** This process scans all your Java/Kotlin files, deletes any code that is never used (dead code elimination), and renames long variables/classes to single letters (e.g., `MainActivity` becomes `a`). This makes the compiled code tiny.
 3. **Resource Shrinking:** This scans your app's asset directories, finds any images, icons, layouts, or files that aren't referenced anywhere in the code, and removes them from the final package.
 4. **Remote Asset Loading (CDN):** 3D models (`.glb/.gltf`) and video files (`.mp4`) are massive. **Never** include them in your project's local folders. Instead, upload them to a web server (like AWS S3 or a CDN) and load them using URLs at runtime.
-

2. Android (Instant Apps) - Step-by-Step Optimization

Android Instant Apps allow users to test your app without installing it. Google enforces a strict **15MB compressed download limit** for Instant Apps.

Step 2.1: Enable Proguard and Resource Shrinking

We need to tell the Android compiler (Gradle) to minify code and delete unused assets during release compilation.

1. Open the file: `android/app/build.gradle` (using your code editor).
2. Find the lines around line 60:

```
def enableProguardInReleaseBuilds = true // <-- Ensure this is set to true
```

3. Scroll down to the `buildTypes` block (usually around line 108) and modify the `release` block to look exactly like this:

```
release {
    signingConfig signingConfigs.debug // Replace with your production
    signing key for Google Play
    minifyEnabled enableProguardInReleaseBuilds
    shrinkResources true // <-- Add this line to delete unused
    assets/drawables
    proguardFiles getDefaultProguardFile("proguard-android.txt"), "proguard-
    rules.pro"
}
```

Step 2.2: Configure CPU Architecture Splitting

We want to generate separate, individual APKs for each CPU type.

1. In the same `android/app/build.gradle` file, find the `splits` block:

```
splits {
    abi {
        enable true // <-- Keep this true to generate split APKs
        reset()
        include "armeabi-v7a", "arm64-v8a" // <-- Remove "x86" and "x86_64"
        universalApk false // <-- Keep this false to prevent generating a
        massive all-in-one APK
    }
}
```

Note: "x86" and "x86_64" are only used for computer simulators. Excluding them saves significant compilation time and space in production.

Step 2.3: Configure Proguard Keep Rules

Because ViroReact uses Java Native Interface (JNI) to link JavaScript to native C++ graphics libraries, Proguard might accidentally delete critical Viro code because it doesn't see it being called from Java. We must add

"Keep Rules" to protect Viro.

1. Open the file: [android/app/proguard-rules.pro](#).
2. Paste the following configuration at the very bottom:

```
# Keep ViroReact Native Methods (Do not let R8 delete JNI connections)
-keepclasseswithmembernames class * {
    native <methods>;
}

# Protect ViroReact Core Classes from deletion/renaming
-keep class com.viro.core.** { *; }
-keep interface com.viro.core.** { *; }
-keep class com.viro.renderer.** { *; }
-keep class com.viromedia.bridge.** { *; }
-keep class com.viromedia.bridge.component.** { *; }

# Ignore warnings from missing internal Google VR / Cardboard libraries
-dontwarn com.google.common.logging.**
-dontwarn com.google.protobuf.**
-dontwarn logs.proto.wireless.performance.mobile.**
```

Step 2.4: How to Build and Check the Real Download Size

When you build the app locally, Windows Explorer shows the **uncompressed** size. But Google Play measures the **compressed** size. Here is how to verify the real download size:

1. Open your terminal, navigate to the [android](#) folder, and compile the release APKs:

```
cd android
.\gradlew.bat assembleRelease
```

2. Once finished, open the output directory in Explorer:

```
explorer.exe app\build\outputs\apk\release\
```

3. Locate [app-arm64-v8a-release.apk](#) (the file for modern devices like Samsung Galaxy, Google Pixel).
4. Right-click on it and compress it into a [.zip](#) archive.
5. **Check the size of the ZIP file.** It will be around **11MB to 13MB**, confirming it safely fits under the 15MB limit!

3. iOS (App Clips) - Step-by-Step Optimization

iOS App Clips are lightweight versions of your iOS app. Apple enforces a **15MB uncompressed limit** on iOS 15/16 (which is raised to **50MB** in iOS 17+).

All iOS optimizations are configured inside **Xcode**.

Step 3.1: Strip Debug Symbols (Removes developer log metadata)

1. Open your project workspace (**ios/MasterARApp.xcworkspace**) in Xcode.
2. Click on your project root in the left sidebar, and select your **App Clip Target**.
3. Go to the **Build Settings** tab at the top.
4. Search for **Strip Debug Symbols During Copy** and set it to **Yes**.
5. Search for **Strip Linked Product** and set it to **Yes**.
6. Search for **Strip Style** and set it to **All Symbols**.

Step 3.2: Set Optimization Level to Smallest Size

This tells the compiler to optimize the output code specifically for the smallest possible binary footprint.

1. In the same **Build Settings** tab, search for **Optimization Level**.
2. Expand it and find the **Release** build configuration.
3. Change it from **-O3** (Fastest) to **Fastest, Smallest [-Os]**.

Step 3.3: Exclude Simulator Architectures

Simulators run on computer chips (**x86_64**). We must exclude them from the production package so they do not add useless weight to the mobile binary.

1. Under **Build Settings**, search for **Build Active Architecture Only**.
2. Set it to **Yes** for the **Release** configuration.
3. Search for **Excluded Architectures**, select **Release**, and add **x86_64** and **i386**.

Step 3.4: Generate the App Thinning Size Report

When you package your app for App Store Connect:

1. In Xcode, go to the top menu: **Product > Archive**.
2. Once archived, click **Distribute App**, select **App Clip**, and proceed.
3. Make sure **App Thinning** is set to **All compatible device variants**. Xcode will output a folder containing a text file named **App Thinning Size Report.txt**.
4. Open that file to read the exact compressed and uncompressed download sizes for every device model (e.g. iPhone 15, iPhone SE).

4. JavaScript & Asset Optimizations (Shared Code)

Since your React Native app shares a JavaScript bundle, optimizations here will automatically make both your iOS and Android apps smaller.

Step 4.1: Enable Hermes JS Engine (Crucial!)

Hermes is Facebook's optimized JavaScript engine. Instead of packaging plain-text JavaScript code, Hermes pre-compiles your JS code into optimized binary bytecode.

- **Android:** Open **android/gradle.properties** and verify this line:

```
hermesEnabled=true
```

- **iOS:** Open `ios/Podfile` and check that Hermes is enabled:

```
use_react_native!(:hermes_enabled => true)
```

Step 4.2: Audit and Remove Large Dependencies

JavaScript packages in `node_modules` can bloat your JS bundle size.

1. **Do not use Moment.js:** It contains massive timezone localization files. Use `dayjs` instead (which is only 2KB).
2. **Avoid full imports:** Instead of importing a whole utility library like this:

```
import _ from 'lodash'; // Imports the entire lodash package (huge)
```

Import only the specific function you need:

```
import cloneDeep from 'lodash/cloneDeep'; // Imports only the cloneDeep function
```

5. Summary Checklist for Release Day

Before you compile your final build for the Google Play Store or Apple App Store, check off this list:

- ☐ **No Local Assets:** Are all your videos (`.mp4`), 3D models (`.glb/.gltf`), and tracking images hosted on a web server? (Verify there are no heavy files in your React Native project directory).
- ☐ **Hermes Engine Enabled:** Verified in both `gradle.properties` (Android) and `Podfile` (iOS).
- ☐ **Proguard Minification Enabled:** `minifyEnabled true` and `shrinkResources true` set in `android/app/build.gradle`.
- ☐ **Xcode Symbols Stripped:** Build settings set to `Strip Debug Symbols = Yes` and `Optimization Level = [-Os]` in Xcode.
- ☐ **Tested on a Real Device:** Have you installed the release build on a physical phone to ensure it starts up and tracks without any minification errors? (Emulators do not support AR).